Hermes: Verified Query Planning over the SPARQL Algebra with Language Agents

Lam Nguyen
Case Western Reserve University
Cleveland, Ohio, USA
ltn18@case.edu

ABSTRACT

Translating natural-language questions into SPARQL lets users query knowledge graphs without learning their schema, and is a basic step in question answering over knowledge graphs. Large language models (LLMs) now write plausible SPARQL, but they write the whole query at once. A query that fails to parse, joins values of incompatible types, or returns nothing is detected only after it is complete, and the repair is another guess. This paper introduces Hermes, a query-planning agent that builds a SPARQL query one operator of the SPARQL algebra at a time, over a typed formalization of the language. (1) We formalize SPARQL query plans as terms of a typed algebra: an RDF schema induces a type for every query variable, and well-formedness and typing rules decide, without evaluating the query, whether a plan is valid. (2) We formulate natural language to SPARQL as verified query planning. Actions are algebra operators; after each one, the agent observes its effect on the graph (the number of solutions, how it changed, a sample of rows, and any error); and a plan is accepted only when it is well typed and satisfies a goal derived from the question. (3) We implement a verifier that rejects ill-typed steps before they reach the graph store, and an LLM planner that drives it. (4) We normalize six benchmarks into one format and implement a scorer that reproduces every published Text2SPARQL 2026 score. All experiments use one small open model, Llama 3.1 8B, run locally. On the Text2SPARQL 2026 corporate benchmark, operator-level planning reaches a set F_1 of 0.094 and a recall of 0.300 and ends with a runnable query on 96 of 100 items, against 0.064 for constraint-guided search and 0.017 for one-shot generation, at a higher model-call budget. [Provisional: the operator-level numbers come from a run whose prompt examples named CK26 content; a rerun is in progress. Pending: the full verified planner, a budget-matched comparison, and DBpedia.]

KEYWORDS

SPARQL, knowledge graphs, natural language interfaces, query algebra, query planning, language agents

PVLDB Reference Format:
Lam Nguyen. Hermes: Verified Query Planning over the SPARQL Algebra with Language Agents. PVLDB, 20(1): XXX-XXX, 2027.
doi:XX.XX/XXX.XX

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/lamng3/hermes.

1 INTRODUCTION

Knowledge graphs such as DBpedia [2] and enterprise graphs hold facts that users want to query in their own words. Natural language interfaces translate a question into a structured query that the graph can execute. For relational data this has a long history, from interactive interfaces that map parse trees onto SQL [11] to recent LLM-based systems [6, 10, 12, 20]. For RDF, question answering has been cast as subgraph matching over the graph [30] and as template decomposition of the question [29], and LLMs now generate SPARQL directly [3, 22, 27].

Existing approaches leave three problems open. **C1. Whole-query generation.** Most LLM-based generators emit a complete query string. A query that fails to parse, joins values of incompatible types, or returns nothing is detected only after it is complete, and the repair is another sample from the same model. **C2. No static guarantees.** A query can parse and still be wrong in ways that the schema already rules out: a literal where the property expects an entity, a filter on a variable that nothing binds, or a projected variable that is neither grouped nor aggregated. Such queries run and quietly return nothing, or an error, and the generator learns little from either. **C3. Search needs a reference.** Constraint-guided search [27] refines a query by adding and removing constraints with Chase and Backchase [4], which gives soundness and completeness guarantees, but relative to a reference answer set that must be supplied or guessed, and over monotone conjunctive constraints, with aggregation and ordering deferred.

Example 1.1. Consider the question "Which department does Ms. Brant belong to?" over the Text2SPARQL corporate graph [1], asked in German. A one-shot generator with an 8B model wrote a query over foaf:Person and foaf:name "Brant", terms the graph does not use, so the query ran and returned nothing. Constraint-guided search kept only the constraint ?d a pv:Department and returned every department. Planning one operator at a time, the agent of Section 4 looked up "Frau Brant" and found the employee Karen Brant, joined ?e pv:memberOf ?d and saw 53 solutions, then filtered on the employee and saw the count fall from 53 to 1: the correct department. Each step was a single algebra operator, and each observation told the agent what that operator did to the answer. [To be replaced with the trace from the corrected rerun; see Section 5.]

Database systems have handled a similar problem for decades. A query optimizer does not guess a whole execution plan; it applies

Table 1: Key notation.

Symbol	Meaning
$\mathcal{G}$; $[\![P]\!]_{\mathcal{G}}$	RDF graph; answers of pattern P on $\mathcal{G}$
I, L, V	IRIs, literals, variables ($?x$)
$O = (C, E)$	ontology: classes C, properties E
dom, *rng*; $\sqsubseteq$	domain and range of a property; subclass order
D; Lit	datatypes; literal types (D and rdfs : Literal)
Q	plan $(T, F, J, X, K, A, H, R, \Pi, \delta, \ell, f)$ (Definition 3.1)
$\bowtie, \sigma, \pi, \gamma, \tau, \lambda$	join, filter, project, group, order, slice (Table 2)
$\beta(Q)$, $\beta^{+}(Q)$	variables bound by Q; with aggregate aliases
Γ_Q	typing context of Q
$O \vdash Q$	Q is valid against O
$g = (\vec{a}, \kappa, c, m)$; $Q \models g$	goal; Q satisfies g
q; a, o, n	question; action, observation, solution count
M, ρ, B	language model, planning policy, step budget

one algebraic rewrite at a time and estimates its effect before keeping it [21], and the quality of those estimates largely decides the quality of the plan [9]. The SPARQL algebra has a compositional semantics [18, 26], so a query can be built and checked the same way. This paper takes that view of natural language to SPARQL.

Contributions. This paper introduces Hermes, a query-planning agent over a typed formalization of the SPARQL algebra.

(1) **A typed SPARQL algebra** (Section 3). We treat query plans as terms of the SPARQL algebra and derive a typing context for their variables from the RDF schema [25]. Well-formedness and typing rules decide, without evaluating a plan, whether it is valid.
(2) **Verified query planning** (Section 3). We formulate natural language to SPARQL as planning: actions are algebra operators, observations report each operator's effect on the graph, and a plan is accepted only when it is well typed and satisfies a goal derived from the question.
(3) **The Hermes system** (Section 4). A verifier rejects ill-typed steps before they reach the graph store, and an LLM planner drives it. Existing generators are policies over the same action space.
(4) **A reproducible evaluation** (Section 5). We normalize six benchmarks and implement a scorer that reproduces all 5,024 published Text2SPARQL 2026 scores. With a small open model (Llama 3.1 8B, run locally) on the corporate benchmark, operator-level planning outperforms one-shot generation and constraint-guided search in F_1 and recall, and almost always ends with a runnable query.

The code, data, configurations, and run outputs are available at the artifact URL above.

2 PRELIMINARIES

Table 1 summarizes the notation used throughout the paper.

RDF graphs and ontologies. Let I and L be disjoint sets of IRIs and literals; blank nodes are treated as IRIs. An RDF graph $\mathcal{G}$ is a finite set of triples $(s, p, o) \in I \times I \times (I \cup L)$. An ontology $O = (C, E)$ over $\mathcal{G}$, its schema, has classes $C \subseteq I$ and properties $E \subseteq I$, a domain $dom(p) \subseteq C$ and a range $rng(p) \subseteq C \cup D$ for each property $p \in E$, where D is a set of datatypes, and the reflexive–transitive closure $\sqsubseteq$ of rdfs : subClassOf [25]. A *type* is a class or a datatype; Lit denotes the datatypes and rdfs : Literal.

Table 2: SPARQL algebra operators, the relational-algebra symbols we write them with, and the Hermes actions that apply them.

Operator	Symbol	Meaning	Action
BGP, Join	$\bowtie$	join triple patterns	triple
Filter	σ_e	keep mappings satisfying e	filter
LeftJoin	$\mathbin{⟕}$	optional extension	optional
Union	$\cup$	either branch	union
Minus	$-$	drop compatible mappings	minus
Extend	$\varepsilon_{?v \leftarrow e}$	bind $?v$ to e	bind
Group, Aggregate	$\gamma_{K;A}$	group and aggregate	group, aggregate
Filter on groups	σ_H	HAVING	having
OrderBy	τ_R	sort	order
Project	π_Π	keep variables	project
Distinct	δ	deduplicate	distinct
Slice	λ_ℓ	LIMIT, OFFSET	limit, offset
ASK form	$\exists$	existence	ask

The SPARQL algebra. Let V be a set of variables, disjoint from $I \cup L$. A triple pattern is a triple over $I \cup L \cup V$. Following Pérez et al. [18] and the SPARQL 1.1 specification [26], a solution mapping μ is a partial function from V to $I \cup L$, two mappings are compatible when they agree on their shared variables, and a graph pattern P evaluates on $\mathcal{G}$ to a multiset of mappings $[\![P]\!]_{\mathcal{G}}$. Patterns are built from basic graph patterns (BGPs) by the operators in Table 2. Evaluating SPARQL graph patterns is PSPACE-complete in general [18], so running a plan to inspect it is not free, which motivates checking what can be checked without running it.

Natural language to SPARQL. Given a question q, a graph $\mathcal{G}$, and its ontology O, the task is to produce a query whose answers on $\mathcal{G}$ match the intended answers of q. Following the Text2SPARQL evaluation [1], answers are compared as sets of values: every value bound to every projected variable counts once, and quality is measured by set precision, recall, and F_1 against the answers of a gold query.

3 TYPED PLANS AND VERIFIED QUERY PLANNING

This section formalizes query plans as typed terms of the SPARQL algebra and states the planning problem that Hermes solves.

Plans. A plan is the logical form of a query, built one operator at a time.

Definition 3.1 (Plan). A plan is a tuple $Q = (T, F, J, X, K, A, H, R, \Pi, \delta, \ell, f)$: a set T of triple patterns; a set F of filter expressions; a set J of groups (ω, T') that combine a set T' of triple patterns with the plan by $\omega \in \{\text{optional}, \text{union}, \text{minus}\}$; extensions X, pairs $(e, ?v)$ of an expression and a variable; group keys $K \subseteq V$; aggregates A, pairs $(\varphi, ?v)$ of an aggregate expression and a variable; group filters H; order keys R; projected variables $\Pi \subseteq V$; a distinct flag δ; a slice ℓ; and a form $f \in \{\text{SELECT}, \text{ASK}\}$.

A plan denotes an algebra term with a fixed operator order, written as in relational algebra and applied right to left:

$$Q \;=\; \lambda_{\ell}\; \delta\; \pi_{\Pi}\; \tau_{R}\; \sigma_{H}\; \gamma_{K;A}\; \sigma_{F}\; \varepsilon_{X}\big(\bowtie T \odot J\big), \tag{1}$$

where $\bowtie T$ joins the triple patterns of T, $\odot J$ applies each group of J with its operator ($\bowtie$ for optional, $\cup$ for union, $-$ for minus), an operator with an empty component is the identity, and δ applies only when the distinct flag is set; an ASK plan replaces the top by $\exists$. This is the order of the SPARQL 1.1 algebra [26]. Rendering is deterministic, so a plan and its query are interchangeable, and we write $[\![Q]\!]_{\mathcal{G}}$ for its answers. An *action* a is a total function from plans to plans that applies one operator of Table 2; the action remove retracts a triple pattern or a filter. Plans are immutable, so a sequence of actions is a path $Q_0 \to Q_1 \to \cdots \to Q_k$ from the empty plan Q_0.

Typing. Let $vars(\cdot)$ denote the variables of a pattern or expression. A *typing context* Γ maps each variable to a set of types. The typing context Γ_Q of a plan Q over an ontology O is the least context closed under the following rules, applied to every triple pattern of T and J:

$$\frac{(?x,\ \mathsf{rdf{:}type},\ c) \quad c \in C}{c \in \Gamma(?x)}\ (\textsc{T-Type})$$

$$\frac{(?x,\ p,\ o) \quad d \in dom(p)}{d \in \Gamma(?x)}\ (\textsc{T-Dom})$$

$$\frac{(s,\ p,\ ?y) \quad r \in rng(p)}{r \in \Gamma(?y)}\ (\textsc{T-Rng})$$

Types are compared up to $\sqsubseteq$.

Definition 3.2 (Well-typed plan). A plan Q is *ill typed* over O if (1) a literal occurs in subject position; (2) a literal is the object of a property p whose range $rng(p)$ contains a class; (3) for some variable, Γ_Q holds both a type in Lit and a class; or (4) a class or property in a namespace that O describes is not in C or E. Otherwise Q is *well typed*.

Two classes in $\Gamma_Q(?x)$ that are incomparable under $\sqsubseteq$ are reported as a warning, not an error, because RDF makes the open-world assumption and O need not declare disjointness.

Definition 3.3 (Well-formed plan). Let $\beta(Q)$ be the variables bound by Q: the variables of T, of the optional and union groups of J, and of the extensions X; the variables of minus groups are not bound. Let $\beta^{+}(Q) = \beta(Q) \cup \{?v \mid (\varphi, ?v) \in A\}$. A plan is *well formed* if $vars(e) \subseteq \beta(Q)$ for every filter and extension e; no extension binds a variable of T or J; $K \subseteq \beta(Q)$; the variables of H, R, and Π are in $\beta^{+}(Q)$; and, when A or K is non-empty, $\Pi \cap \beta(Q) \subseteq K$.

A plan that is both well typed and well formed over O is *valid*, written $O \vdash Q$. Validity is decided from Q and O alone, without evaluating Q on $\mathcal{G}$.

Goals. A question constrains the shape of its answer before any query exists: "which department" asks for entities of one class, "how many" for a number, "is there" for a Boolean.

Definition 3.4 (Goal). A goal is $g = (\vec{a}, \kappa, c, m)$: answer variables $\vec{a}$, a kind $\kappa \in \{\mathsf{entity}, \mathsf{literal}, \mathsf{number}, \mathsf{boolean}\}$, an optional type c, and a cardinality $m \in \{\mathsf{one}, \mathsf{many}\}$. A valid plan Q *satisfies* g, written $Q \vDash g$, if: for $\kappa = \mathsf{boolean}$, $f = \mathsf{ASK}$; otherwise $f = \mathsf{SELECT}$, $\vec{a} \subseteq \beta^{+}(Q)$, the projection is exactly $\vec{a}$, a number is witnessed by an aggregate or a numeric datatype in Γ_Q, an entity has no type in Lit, and, if c is given, some type in Γ_Q is subsumed by c. The cardinality $m = \mathsf{one}$ is checked after execution: $|[\![Q]\!]_{\mathcal{G}}| = 1$.

The goal is derived from the question by the planner, so it can be wrong; satisfaction guarantees that the plan has the shape its goal asks for, not that the goal reflects the question.

The planning problem.

Definition 3.5 (Verified query planning). Given a question q, a graph $\mathcal{G}$, its ontology O, and a step budget B, a policy ρ states a goal g and applies actions $a_1, \ldots, a_k$ with $k \le B$, producing plans $Q_1, \ldots, Q_k$. After each action it receives an observation $o_i = (ok_i, n_i, n_{i-1}, rows_i, err_i)$: whether Q_i is valid and runs, its number of solutions $n_i = |[\![Q_i]\!]_{\mathcal{G}}|$, the previous count, a sample of rows, and an error message. An action whose plan is not valid is rejected and not executed. The output Q_k must satisfy $O \vdash Q_k$ and $Q_k \vDash g$; among such outputs, the objective is the set F_1 of $[\![Q_k]\!]_{\mathcal{G}}$ against the intended answers of q.

Example 3.6. Take the question "Which US city hosted the most Winter Olympics?" over a small graph of Olympic host cities. The planner first states the goal $g = (?city, \mathsf{entity}, \mathsf{ex{:}City}, \mathsf{one})$. The query it builds is

```
SELECT ?city WHERE {
  ?city ex:hosted ?e .  ?city a ex:City .
  ?city ex:country ex:USA .
  ?e ex:season "Winter" . }
GROUP BY ?city ORDER BY DESC(COUNT(?e))
LIMIT 1
```

and, as a term of Equation (1),

$$\lambda_1\ \pi_{?city}\ \tau_{\downarrow n}\ \gamma_{?city;\, n \leftarrow \mathsf{COUNT}(?e)}\ \big(t_1 \bowtie t_2 \bowtie t_3 \bowtie t_4\big),$$

where

$$t_1 = (?city,\ \mathsf{ex{:}hosted},\ ?e),$$
$$t_2 = (?city,\ \mathsf{a},\ \mathsf{ex{:}City}),$$
$$t_3 = (?city,\ \mathsf{ex{:}country},\ \mathsf{ex{:}USA}),$$
$$t_4 = (?e,\ \mathsf{ex{:}season},\ \text{"Winter"}).$$

Figure 1 shows the plan as an operator tree, built bottom up by ten actions, with the number of solutions the explorer observed after each one. Joining t_3 and t_4 cuts 19 solutions to 8 and then 4; grouping leaves 3 cities; the slice leaves 1, Lake Placid. The verifier keeps two goals open until the end: $?city : \mathsf{ex{:}City}$ until t_2 states the type, since this graph declares no domain for $\mathsf{ex{:}hosted}$, and a single projected answer until the slice and the projection. finish is accepted only after the last action.

The intended answers are unknown during planning, so the objective cannot be optimized directly; the observations are the evidence the policy has. Three properties make this action space suitable for planning.

Proposition 3.7 (Monotone refinement). *Let Q be a plan with $J = A = \emptyset$ and no slice, and let Q' apply* triple *or* filter *to Q. Then every mapping in $[\![Q']\!]_{\mathcal{G}}$, restricted to $\beta(Q)$, is a mapping in $[\![Q]\!]_{\mathcal{G}}$.*

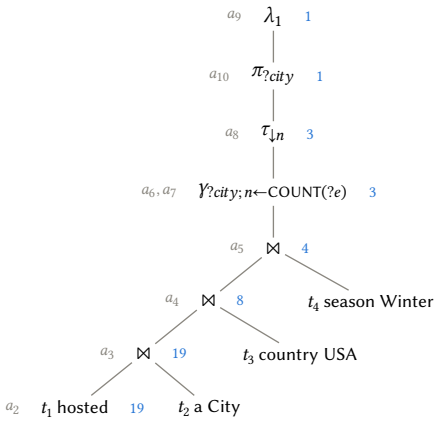


Figure 1: The plan of Example 3.6 as an operator tree. Gray: the action that added each operator (a_1 states the goal). Blue: solutions observed after that action.

Proof sketch. A mapping of the join of Q with a triple pattern is the union of compatible mappings of Q and of the pattern, so its restriction to $\beta(Q)$ is a mapping of Q. Filtering only removes mappings. □

Proposition 3.7 makes the count in an observation informative: within the conjunctive fragment, a new constraint can only narrow the answers seen through the variables already present, so a drop from 53 solutions to 1 (Example 1.1) is evidence that the constraint is selective.

Proposition 3.8 (Static soundness). *If the verifier of Section 4 accepts Q, then $O \vdash Q$.*

Proposition 3.9 (Checking cost). *After a one-time closure of $\sqsubseteq$, deciding $O \vdash Q$ and $Q \models g$ takes time linear in the size of Q, up to the cost of set lookups in O.*

Proofs of Propositions 3.8 and 3.9 are in Appendix A.

4 THE HERMES PLANNER

Hermes has four components: an entity linker, an explorer that applies actions and observes their effect, a verifier that decides validity and goal satisfaction, and a planner that chooses actions. Algorithm 1 in Appendix B gives the loop.

Actions. The action space is the algebra of Table 2, plus three kinds of auxiliary actions. *Graph reads* leave the plan unchanged: lookup returns entities whose label contains a string, and inspect lists the predicates around an entity or used by the instances of a class, with counts and an example value. *Control* actions are goal, which states g; undo, which restores the previous plan; and finish. Each action is a JSON object, so a planner emits one action per turn and the explorer parses it into a typed command.

Observations. After a plan-changing action, the explorer renders the new plan, checks that it parses, asks the verifier for validity, and only then executes two queries on the graph store: the number of solutions, as a COUNT over the plan used as a subquery, and a sample of rows under a LIMIT (five by default). The observation reports the count, the previous count when it changed, the sample, and any error. An action whose plan does not parse, is not valid, or fails to run is not committed. preview returns the same observation without committing, the analogue of costing a rewrite before choosing it. Executions are memoized, so revisiting a plan costs nothing.

Verifier. The verifier computes Γ_Q from the schema, checks the conditions of Section 3, and returns errors, warnings, the typing context, and the conditions of g that Q does not yet satisfy (the *open goals*). Error messages name the offending term and, for an unknown class or property, suggest the closest term in the schema. Algorithm 2 in Appendix B gives the procedure.

Planner. The policy ρ is a language model M prompted, at every turn, with the action catalogue, the vocabulary of O, the entities whose labels occur in the question, the history of actions and observations, and the current plan as SPARQL. With the verifier enabled, the prompt also shows the typing context and the open goals, and finish is refused while an error or an open goal remains. An action identical to the previous one is not executed. The loop ends on finish or after the step budget B (15), and returns the last committed plan that runs.

Other policies in the same space. Existing generators are policies over these actions. One-shot generation is a single macro action that sets every field of the plan at once. Constraint-guided search [27] is a beam search over remove and triple: a Chase phase removes constraints until the plan covers a reference answer set, and a Backchase phase adds them back until it returns only reference answers [4]. Placing them in one space lets the evaluation hold the graph store, the scorer, and the model fixed while varying only the policy.

Implementation. Hermes is written in Python. Graph stores share one interface that returns SPARQL 1.1 JSON results, so the same planner runs on local RDF files loaded into Oxigraph [16] or rdflib, or on a remote SPARQL endpoint. Model completions are cached on disk, so a rerun replays the same outputs, and each run records the model, the number of model calls and graph executions, and the commit it started from.

5 EXPERIMENTAL STUDY

We evaluate Hermes on three questions. **[RQ1] Effectiveness**: how accurate is operator-level planning against one-shot generation and constraint-guided search, at the same model and on the same graph store? **[RQ2] Components**: what do the verifier, the execution feedback, and the reference answers contribute? **[RQ3] Cost**: how many model calls, graph executions, and how much time does each method spend? All configurations and run outputs are in the artifact.

5.1 Experimental Setup

Benchmarks. We normalize six benchmarks into one format (Table 3): every question and language becomes one item with its gold query, feature tags (*e.g.*,COUNT, GROUP, ORDER, ASK), and gold answers stored as in the official Text2SPARQL scorer. Sources are pinned to commits and checksums. This paper reports CK26 and DB26, the two tracks of the Text2SPARQL 2026 challenge [1]. CK26

Table 3: Normalized benchmarks. Items are question–language pairs.

Benchmark	Questions	Items	Graph
CK26 [1]	50	100	corporate, local
DB26 [1]	50	100	DBpedia, endpoint
CK25 [23]	50	50	corporate, local
DB25 [23]	100	200	DBpedia, endpoint
QALD-10 [24]	394	1,564	Wikidata, live
QALD-9-plus [17]	150	450	DBpedia, live

asks 50 questions in English and German over a corporate graph of products, suppliers, and employees (13 classes, 30 properties, about 26.6k triples), loaded locally; DB26 asks 50 questions in English and Spanish over DBpedia [2], queried at the organizers' endpoint. Each track has 100 items.

Data caveats. Three properties of the data shape what a number means. (1) Forty-nine of the fifty CK26 gold queries appear verbatim in CK25, on the same graph, so neither is a development set for the other; no method here uses either for examples or tuning. (2) The CK gold answers were computed on the Tentris engine. Rerunning the gold queries on Oxigraph reproduces 43 of the 50; the other seven differ in tie-breaking under LIMIT, decimal formatting, or the non-standard cast xsd:int(). We report results on all items and on the 86 items whose gold reproduces. All 50 DB26 gold queries reproduce on the organizers' endpoint. (3) The official scorer has two quirks for ASK and ordered questions, described in Appendix C; we report the official metric and a strict variant that corrects them.

Metrics. We report set precision (P), recall (R), and F_1, and the challenge's ranking metric F_{ndcg}, which uses nDCG on questions whose order matters and set F_1 elsewhere. Our scorer reproduces all 5,024 per-question and average scores published for the 2026 challenge (Appendix C), so our numbers are on the leaderboard's scale.

Methods. *OneShot* prompts a single query from the schema vocabulary, declares any known prefix the draft omits, and makes one repair call if the draft does not parse. *CGS* is our reimplementation of constraint-guided search with Chase and Backchase [27]. It follows the published three stages, with two documented deviations: queries are rendered from constraints deterministically rather than by a second model, and candidates are scored by comparing answer sets with the reference answers rather than by a model. It uses two model calls per question, one for the constraint table and one for the reference answers, and beam width 5. *Hermes*$^-$ is the planner of Section 4 with the verifier disabled: execution feedback, no typing, no goal. *Hermes* is the full verified planner. [Pending: Hermes run on CK26 and DB26.]

Prompt overlap. The Hermes$^-$ run reported in this draft used an earlier planner prompt whose action examples named one CK26 entity (an employee in item 1) and two CK26 properties (memberOf and phone). The examples now use an unrelated domain (Figure 4), and a rerun with the corrected prompt is in progress; until it finishes, every Hermes$^-$ number in this section is provisional. The OneShot and CGS prompts use generic or unrelated examples.

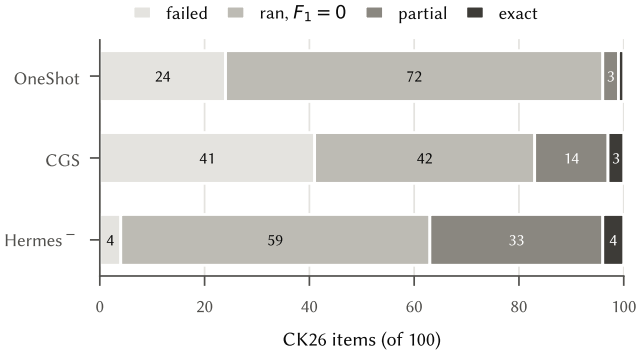


Figure 2: Outcome of each CK26 item (Llama 3.1 8B): no runnable query, a query with $F_1 = 0$, a partial answer, or the exact answer.

Model. Every experiment in this paper uses one small open model: Llama 3.1 8B [7] with 4-bit quantization (Q4_0), served locally by Ollama [14] at temperature 0. We chose it so that every result can be reproduced on a laptop, with no API cost, and so that differences between methods are not masked by a model that can answer from memory. The published Text2SPARQL entries use larger models, so our numbers are a lower bound for each method, not its ceiling. Hermes takes the model as a parameter, local or behind any OpenAI-compatible API, so the same runs extend to other open models such as Qwen2.5 and Qwen3 [19, 28] and to API models such as GPT-4 [15]. [Pending: results with further small and API models.]

Implementation. All methods use this model, the same graph stores, and the same scorer. The planners have a step budget of $B = 15$. Experiments run on a single Apple-silicon laptop.

5.2 RQ1: Effectiveness

Table 4 reports the finished runs. On CK26, Hermes$^-$ reaches the highest F_1 (0.094) and recall (0.300), against 0.064 and 0.142 for CGS and 0.017 and 0.015 for OneShot. It answers 4 items exactly and 37 with non-zero F_1 (Figure 2). The ranking holds in both languages: F_1 is 0.098 (English) and 0.091 (German) for Hermes$^-$, 0.048 and 0.081 for CGS, and 0.007 and 0.027 for OneShot. Restricting to the 86 items whose gold reproduces changes no F_{ndcg} by more than 0.004. On DB26, where the DBpedia vocabulary is too large to show to the model, OneShot reaches 0.017, as on CK26. [Pending: CGS, Hermes$^-$, and Hermes on DB26.]

For scale, Table 5 places our runs among the published 2026 entries on CK26, which range from 0.157 to 0.832 in F_{ndcg} [1, 22]. This is not a paired comparison: those systems were scored by the organizers on Tentris, and their models are not controlled here. Every method in Table 4 uses an 8B local model and stays below the weakest entry.

Table 6 breaks CK26 down by the feature tags of the gold queries. Operator-level planning gains most on plain selections and filters. No method answers aggregation, slicing, or ASK questions: every F_1 on COUNT, GROUP, and LIMIT is at most 0.03, and all three

Table 4: Effectiveness (official metrics; strict is set F_1 under the corrected scorer). Model: Llama 3.1 8B. The gold-oracle row is an ablation that uses gold answers.

	Method	P	R	F_1	F_{ndcg}	Strict
CK26	OneShot	0.036	0.015	0.017	0.018	0.017
	CGS	0.061	0.142	0.064	0.064	0.064
	Hermes$^-$	**0.101**	**0.300**	**0.094**	**0.094**	**0.094**
	Hermes	–	–	–	–	–
	CGS, gold oracle	0.082	0.229	0.086	0.086	0.086
DB26	OneShot	0.038	0.011	0.017	0.017	0.017

Table 5: CK26 F_{ndcg} of the published Text2SPARQL 2026 entries, scored by the organizers, and of our runs (shaded, 8B local model).

Entry	F_{ndcg}	Entry	F_{ndcg}
SPARQL-LLM [22]	**0.832**	IRIS	0.217
INFAI-ETI-A	0.633	LIBER-AI-QWEN	0.206
ADFR	0.513	INFAI-ETI-B	0.157
INFAI-ETI-C	0.452	Hermes$^-$	0.094
LIBER-AI-CLAUDE	0.335	CGS	0.064
		OneShot	0.018

Table 6: CK26 set F_1 by feature tag of the gold query; a question can carry several tags, and n counts items. Best per row in bold.

Tag	n	OneShot	CGS	Hermes$^-$
SELECT	94	0.018	0.068	**0.100**
FILTER	10	0.007	**0.293**	0.290
ORDER	30	0.012	**0.025**	0.022
SUBSELECT	8	0	0.011	**0.022**
GROUP	18	0	**0.019**	0.012
COUNT	16	0	**0.016**	0.013
LIMIT	26	0	**0.029**	0.002
ASK	6	0	0	0

score 0 on the six ASK items. These are exactly the questions whose answer shape (a number, a single value, a Boolean) a goal states before the query exists, which is what the verified planner is designed to enforce.

5.3 RQ2: Components

Reference answers. CGS needs a reference answer set. The gold-oracle ablation replaces the LLM's reference answers with the gold answers and replays the same cached constraint tables, so it changes only the reference. F_1 rises from 0.064 to 0.086 and recall from 0.142 to 0.229. It stays below Hermes$^-$, which uses no reference answers, because 41 items fail before the search starts: the model's constraint table matches nothing in the graph, and no reference can repair that.

Verifier. [Pending: Hermes against Hermes$^-$ at equal step budget; kernel rejections per item; effect of goal-gated finish.]

Feedback signals. [Pending: ablation of the observation, removing in turn the count, the change in count, the row sample, and the error message.]

Table 7: Time and cost per run on CK26 (100 items, Llama 3.1 8B on one laptop). Wall is the whole run; p50 and p95 are seconds per item. Errors are items with no query or a query that fails to run.

Method	Calls	Exec.	Wall (min)	p50	p95	Errors
OneShot	116	136	**6.2**	**3.0**	**8.5**	24
CGS	198	667	35.6	15.1	58.1	41
gold oracle	(cached)	4,051	37.2	0.0	0.8	41
Hermes$^-$	1,451	422	40.4	24.4	34.6	**4**
Hermes	–	–	–	–	–	–

5.4 RQ3: Time and Cost

All runs execute on one laptop (Apple M2 Pro, 16 GB) with the model served locally, so no run pays for API calls; cost is model calls, graph executions, and time. Table 7 reports them per run; Figure 3(a) sets accuracy against model calls, and Figure 3(b) shows the time of every item.

Model calls. Hermes$^-$ ends with a runnable query on 96 of 100 items, against 59 for CGS and 76 for OneShot, but it spends 1,451 model calls (14.5 per item, 13.9 steps on average) against 198 for CGS and 116 for OneShot. The comparison in Table 4 is therefore not at equal model budget, and does not by itself show that execution feedback beats generation with repair. [Pending: budget-matched comparison, *e.g.*, CGS or OneShot with self-consistency at 15 calls per item.]

Time. OneShot answers 100 items in 6.2 minutes (3.0 s median per item). CGS takes 35.6 minutes (15.1 s median, 58.1 s at the 95th percentile): its beam search runs 667 graph executions, and its slowest items are those with large constraint tables. Hermes$^-$ takes 40.4 minutes (24.4 s median, 34.6 s at the 95th percentile), and its time is the most predictable of the three: every item falls between about 6 and 50 s (Figure 3(b)), because the 15-step budget bounds each one. Graph executions stay cheap for the planner: memoization keeps it at 422, below the 667 of CGS. The gold-oracle ablation shows the risk of search without a time bound: with every model call replayed from the cache, the median item takes under a second, yet the run takes 37.2 minutes, because one item spends 1,919 s (86% of the run) on 81 graph executions, among them a single very expensive candidate query. The local store has no query timeout; bounding each execution is a simple guard that both search and planning need. [Pending: input and output tokens and the split of time between the model and the graph store, which runs record from the corrected rerun onward.]

5.5 Case Study

This item is the one the earlier prompt named; the trace will be replaced by one from the corrected rerun. Table 8 is the trace of Hermes$^-$ on the German version of Example 1.1. Two observations carry the answer. Joining ?e pv:memberOf ?d shows 53 employee–department pairs, so the agent knows the join is correct but unselective; the filter on the employee found by lookup narrows it to one solution, which is the gold answer. The trace also shows the limits of an unverified planner: two of its eight actions are lookups that return nothing.

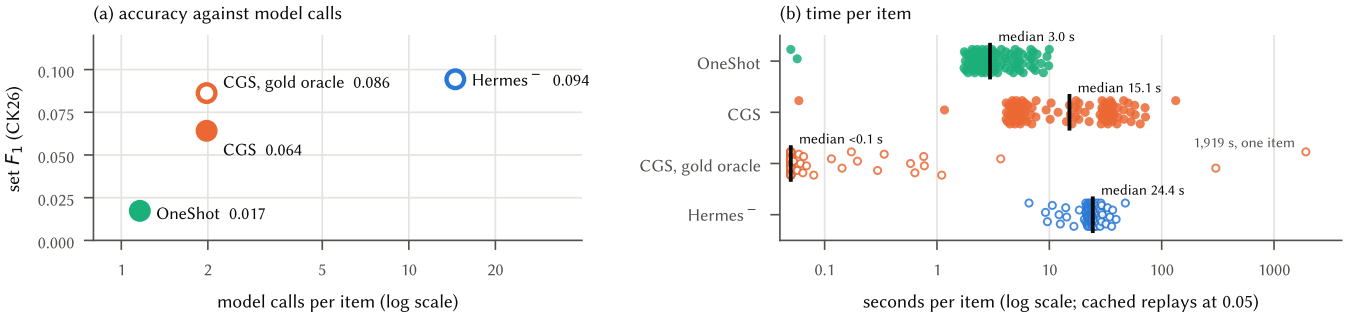


Figure 3: Cost on CK26 (Llama 3.1 8B, one laptop). (a) Set F_1 against model calls per item. (b) Seconds per item, one dot per item, with the median as a bar; items replayed from the cache are drawn at 0.05 s. Hollow markers are ablations: CGS with gold reference answers, and Hermes without the verifier.

Table 8: Trace of Hermes⁻ on "Zu welcher Abteilung gehört Frau Brant?" (CK26, item 1, German).

Action	Observation
lookup "Frau Brant"	2 entities, first: employee Karen Brant
lookup "Abteilung"	no entity
inspect pv : Department	13 predicates
triple ?e pv : memberOf ?d	53 solutions
project ?d	53 solutions
lookup "Zu welcher Abteilung"	no entity
filter ?e = Karen Brant	53 → 1 solution
finish	—

6 RELATED WORK

Natural language interfaces to data. Natural language interfaces to relational databases map a parsed question onto SQL, and NaLIR made the mapping interactive by asking the user to resolve ambiguities [11]. For RDF, gAnswer casts question answering as subgraph matching between a semantic query graph and the knowledge graph [30], and template decomposition answers complex questions by composing binary templates mined from the graph [29]. GraphLingo explores domain knowledge graphs in natural language by synchronizing them with an LLM [8]. Benchmarks for question answering over RDF include LC-QuAD 2.0 [5], QALD-9-plus [17], QALD-10 [24], and the Text2SPARQL challenges [1, 23].

LLM-based text-to-SQL. With LLMs, text-to-SQL has moved to prompting and fine-tuning. DAIL-SQL studies how question representation and example selection shape LLM prompts [6], PURPLE retrieves demonstrations that match the logical operators a question needs [20], CodeS pre-trains open models for SQL generation [12], and NL2SQL360 evaluates these systems along many dimensions [10]. These methods emit a whole query and improve it by better context or better models; none types the query against the schema before it runs.

LLM-based text-to-SPARQL. SPARQL-LLM, the strongest entry of the 2026 Text2SPARQL challenge, generates SPARQL from natural-language questions in real time [22]. ARUQULA explores the graph with tools in a ReAct loop before writing a query [3]. UniQGen extracts a table of constraints from the question and the graph and refines a query with Chase and Backchase against reference answers from an LLM, rendering it to SPARQL or Cypher [27]. Hermes differs from these in its unit of generation: an operator of the SPARQL algebra rather than a query string or a constraint table, checked against the schema before execution and observed after it.

Query optimization and SPARQL semantics. Cost-based optimizers choose among algebraically equivalent plans by estimating the effect of each operator [21], and cardinality estimates dominate the quality of the chosen plan [9]. Chase and Backchase reformulates queries under constraints and minimizes them [4]. Pérez et al. gave SPARQL a compositional semantics and characterized the complexity of its evaluation [18]. Hermes borrows the optimizer's step, one operator with an observed effect, and the algebra's semantics, and uses them to generate a query rather than to rewrite one.

7 CONCLUSION

We have presented Hermes, which treats natural language to SPARQL as verified query planning over a typed SPARQL algebra. Plans are typed against the RDF schema and checked without execution; actions are algebra operators whose effect on the graph is observed; and a plan is accepted only when it is valid and satisfies a goal derived from the question. On the Text2SPARQL 2026 corporate benchmark with a small open model (Llama 3.1 8B), operator-level planning outperforms one-shot generation and constraint-guided search and almost always ends with a runnable query, at a higher call budget. All methods fail on aggregation and ASK questions, whose answer shape a goal can state in advance. Next steps are the evaluation of the full verified planner, a budget-matched comparison, ablations of the observation, the DBpedia and Wikidata benchmarks, and the proofs in Appendix A. [Update once the pending runs finish.]

REFERENCES

[1] AKSW. 2026. The Text2SPARQL Challenge 2026: Datasets, Results, and Evaluation Client. Challenge site and result repository. https://github.com/AKSW/text2sparql.aksw.org

[2] Sören Auer, Christian Bizer, Georgi Kobilarov, Jens Lehmann, Richard Cyganiak, and Zachary Ives. 2007. DBpedia: A Nucleus for a Web of Open Data. In *The Semantic Web, 6th International Semantic Web Conference (ISWC) (Lecture Notes in Computer Science)*. Springer, Berlin, Heidelberg, 722–735. https://doi.org/10.1007/978-3-540-76298-0_52

[3] Felix Brei, Lorenz Bühmann, Johannes Frey, Daniel Gerber, Lars-Peter Meyer, Claus Stadler, and Kirill Bulert. 2025. ARUQULA – An LLM based Text2SPARQL Approach using ReAct and Knowledge Graph Exploration Utilities. In *Proceedings of the First International TEXT2SPARQL Challenge 2025, co-located with Text2KG at ESWC 2025 (CEUR Workshop Proceedings)*, Vol. 4094. CEUR-WS.org, Aachen, Germany, 49–63. https://ceur-ws.org/Vol-4094/

[4] Alin Deutsch, Lucian Popa, and Val Tannen. 1999. Physical Data Independence, Constraints, and Optimization with Universal Plans. In *Proceedings of the 25th International Conference on Very Large Data Bases (VLDB)*. Morgan Kaufmann, San Francisco, CA, USA, 459–470.

[5] Mohnish Dubey, Debayan Banerjee, Abdelrahman Abdelkawi, and Jens Lehmann. 2019. LC-QuAD 2.0: A Large Dataset for Complex Question Answering over Wikidata and DBpedia. In *The Semantic Web – ISWC 2019 (Lecture Notes in Computer Science)*. Springer, Cham, 69–78. https://doi.org/10.1007/978-3-030-30796-7_5

[6] Dawei Gao, Haibin Wang, Yaliang Li, Xiuyu Sun, Yichen Qian, Bolin Ding, and Jingren Zhou. 2024. Text-to-SQL Empowered by Large Language Models: A Benchmark Evaluation. *Proceedings of the VLDB Endowment* 17, 5 (2024), 1132–1145. https://doi.org/10.14778/3641204.3641221

[7] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, et al. 2024. The Llama 3 Herd of Models. arXiv:2407.21783. https://arxiv.org/abs/2407.21783

[8] Duy Le, Kris Zhao, Mengying Wang, and Yinghui Wu. 2024. GraphLingo: Domain Knowledge Exploration by Synchronizing Knowledge Graphs and Large Language Models. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE, Piscataway, NJ, USA, 5477–5480. https://doi.org/10.1109/ICDE60146.2024.00432

[9] Viktor Leis, Andrey Gubichev, Atanas Mirchev, Peter Boncz, Alfons Kemper, and Thomas Neumann. 2015. How Good Are Query Optimizers, Really? *Proceedings of the VLDB Endowment* 9, 3 (2015), 204–215. https://doi.org/10.14778/2850583.2850594

[10] Boyan Li, Yuyu Luo, Chengliang Chai, Guoliang Li, and Nan Tang. 2024. The Dawn of Natural Language to SQL: Are We Fully Ready? *Proceedings of the VLDB Endowment* 17, 11 (2024), 3318–3331. https://doi.org/10.14778/3681954.3682003

[11] Fei Li and H. V. Jagadish. 2014. Constructing an Interactive Natural Language Interface for Relational Databases. *Proceedings of the VLDB Endowment* 8, 1 (2014), 73–84. https://doi.org/10.14778/2735461.2735468

[12] Haoyang Li, Jing Zhang, Hanbing Liu, Ju Fan, Xiaokang Zhang, Jun Zhu, Renjie Wei, Hongyan Pan, Cuiping Li, and Hong Chen. 2024. CodeS: Towards Building Open-source Language Models for Text-to-SQL. *Proceedings of the ACM on Management of Data (SIGMOD)* 2, 3 (2024), 1–28. https://doi.org/10.1145/3654930

[13] National Institute of Standards and Technology. 2024. trec_eval. Information retrieval evaluation software. https://github.com/usnistgov/trec_eval

[14] Ollama contributors. 2026. Ollama. Software for running language models locally. https://github.com/ollama/ollama

[15] OpenAI. 2023. GPT-4 Technical Report. arXiv:2303.08774. https://arxiv.org/abs/2303.08774

[16] Oxigraph contributors. 2026. Oxigraph. RDF graph database. https://github.com/oxigraph/oxigraph

[17] Aleksandr Perevalov, Dennis Diefenbach, Ricardo Usbeck, and Andreas Both. 2022. QALD-9-plus: A Multilingual Dataset for Question Answering over DBpedia and Wikidata Translated by Native Speakers. In *2022 IEEE 16th International Conference on Semantic Computing (ICSC)*. IEEE, Piscataway, NJ, USA, 229–234. https://doi.org/10.1109/ICSC52841.2022.00045

[18] Jorge Pérez, Marcelo Arenas, and Claudio Gutierrez. 2009. Semantics and Complexity of SPARQL. *ACM Transactions on Database Systems* 34, 3 (2009), 16:1–16:45. https://doi.org/10.1145/1567274.1567278

[19] Qwen Team. 2024. Qwen2.5 Technical Report. arXiv:2412.15115. https://arxiv.org/abs/2412.15115

[20] Tonghui Ren, Yuankai Fan, Zhenying He, Ren Huang, Jiaqi Dai, Can Huang, Yinan Jing, Kai Zhang, Yifan Yang, and X. Sean Wang. 2024. PURPLE: Making a Large Language Model a Better SQL Writer. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE, Piscataway, NJ, USA, 15–28. https://doi.org/10.1109/ICDE60146.2024.00009

[21] Patricia G. Selinger, Morton M. Astrahan, Donald D. Chamberlin, Raymond A. Lorie, and Thomas G. Price. 1979. Access Path Selection in a Relational Database Management System. In *Proceedings of the 1979 ACM SIGMOD International Conference on Management of Data*. ACM, New York, NY, USA, 23–34. https://doi.org/10.1145/582095.582099

[22] Panayiotis Smeros, Vincent Emonet, Ruijie Wang, Ana-Claudia Sima, and Tarcisio Mendes de Farias. 2026. SPARQL-LLM: Real-Time SPARQL Query Generation from Natural Language Questions. *ACM Transactions on the Web* (2026). https://doi.org/10.1145/3847195

[23] Sebastian Tramp, Marcos Gôlo, Edgard Marx, and Paulo Viviurka do Carmo (Eds.). 2025. *Proceedings of the First International TEXT2SPARQL Challenge 2025, co-located with Text2KG at ESWC 2025*. CEUR Workshop Proceedings, Vol. 4094. CEUR-WS.org, Aachen, Germany. https://ceur-ws.org/Vol-4094/

[24] Ricardo Usbeck, Xi Yan, Aleksandr Perevalov, Longquan Jiang, Julius Schulz, Angelie Kraft, Cedric Moeller, Junbo Huang, Jan Reineke, Axel-Cyrille Ngonga Ngomo, Muhammad Saleem, and Andreas Both. 2023. QALD-10 — The 10th Challenge on Question Answering over Linked Data. Semantic Web journal, submission 3357; dataset at https://github.com/KGQA/QALD-10. https://www.semantic-web-journal.net/node/3357

[25] W3C RDF Working Group. 2014. RDF Schema 1.1. W3C Recommendation, 25 February 2014. https://www.w3.org/TR/rdf-schema/

[26] W3C SPARQL Working Group. 2013. SPARQL 1.1 Query Language. W3C Recommendation, 21 March 2013. https://www.w3.org/TR/sparql11-query/

[27] Mengying Wang, Nicolaas Jedema, Rahul Pandey, RaviKiran Krishnan, Jens Lehmann, and Yinghui Wu. 2026. Graph Query Generation with Constraint-guided Large Language Agents. arXiv:2605.00845. https://arxiv.org/abs/2605.00845

[28] An Yang, Anfeng Li, Baosong Yang, et al. 2025. Qwen3 Technical Report. arXiv:2505.09388. https://arxiv.org/abs/2505.09388

[29] Weiguo Zheng, Jeffrey Xu Yu, Lei Zou, and Hong Cheng. 2018. Question Answering over Knowledge Graphs: Question Understanding via Template Decomposition. *Proceedings of the VLDB Endowment* 11, 11 (2018), 1373–1386. https://doi.org/10.14778/3236187.3236192

[30] Lei Zou, Ruizhe Huang, Haixun Wang, Jeffrey Xu Yu, Wenqiang He, and Dongyan Zhao. 2014. Natural Language Question Answering over RDF: A Graph Data Driven Approach. In *Proceedings of the 2014 ACM SIGMOD International Conference on Management of Data*. ACM, New York, NY, USA, 313–324. https://doi.org/10.1145/2588555.2610525

Algorithm 1: Verified query planning

Input: question q, graph $\mathcal{G}$, ontology O, policy ρ, step budget B
Output: a plan Q

$Q \leftarrow Q_0$; $g \leftarrow \bot$; $h \leftarrow [\,]$;
for $i \leftarrow 1$ **to** B **do**
  $a \leftarrow \rho(q, O, Q, g, h)$;
  if *a is* goal **then**
    $g \leftarrow a.g$; $o \leftarrow$ Verify(Q, g);
  else if *a is* lookup *or* inspect **then**
    $o \leftarrow$ read $\mathcal{G}$;
  else if *a is* finish **then**
    if Verify(Q, g) *has no errors and no open goals* **then return** Q;
    $o \leftarrow$ refusal with the open goals;
  else
    $Q' \leftarrow a(Q)$; $o \leftarrow$ Verify(Q', g);
    if *o has no errors* **then**
      $o \leftarrow o \cup \{|[\![Q']\!]_{\mathcal{G}}|, \text{sample}\}$;
      if Q' *runs* **then** $Q \leftarrow Q'$;
  append (a, o) to h;
return the last plan in h that runs;

Algorithm 2: Verify(Q, g)

Input: plan Q, goal g (possibly $\bot$), ontology O
Output: errors *Err*, typing context Γ_Q, open goals *Open*

Err $\leftarrow \emptyset$; $\Gamma_Q \leftarrow \emptyset$;
foreach *triple pattern* (s, p, o) *of* T *and* J **do**
  if $p \notin E$, *or a class not in* C, *lies in a namespace* O *describes* **then** add to *Err*;
  if s *is a literal, or* o *is a literal and* $rng(p)$ *has a class* **then** add to *Err*;
  apply T-Type, T-Dom, and T-Rng to extend Γ_Q;
foreach *variable with a type in* Lit *and a class in* Γ_Q **do** add to *Err*;
check the conditions of Definition 3.3 on $\beta(Q)$ and $\beta^+(Q)$, adding violations to *Err*;
Open $\leftarrow$ the conditions of $Q \vDash g$ (Definition 3.4) that fail;
return $(Err, \Gamma_Q, Open)$;

APPENDIX

A. Theorems and Proofs

Proposition 3.7 (Monotone refinement).

Proof. Let Q be a plan with $J = A = \emptyset$ and no slice, and let $\mu' \in [\![Q']\!]_{\mathcal{G}}$. If Q' adds a triple pattern t, then by the semantics of join $\mu' = \mu \cup \nu$ for some compatible $\mu \in [\![Q]\!]_{\mathcal{G}}$ and $\nu \in [\![t]\!]_{\mathcal{G}}$; μ is defined exactly on $\beta(Q)$, so the restriction of μ' to $\beta(Q)$ is μ. If Q' adds a filter e, then $[\![Q']\!]_{\mathcal{G}}$ is the set of mappings of $[\![Q]\!]_{\mathcal{G}}$ that satisfy e. In both cases the restriction of μ' to $\beta(Q)$ is in $[\![Q]\!]_{\mathcal{G}}$. □

Proposition 3.8 (Static soundness). [Proof to be written: by induction on the structure of the plan, following Algorithm 2, showing that each condition of the definitions of well-typed and well-formed plans is checked.]

Proposition 3.9 (Checking cost). [Proof to be written: one pass over the triple patterns with constant-time schema lookups, one pass over expressions, and a precomputed subclass closure.]

B. Algorithms and Analysis

Algorithm 1 is the planning loop of Section 4; Algorithm 2 is the verifier.

[Complexity analysis of Algorithms 1 and 2: to be written with Proposition 3.9.]

C. Evaluation Details

Scorer parity. The official Text2SPARQL scorer flattens every value of every variable into one set and scores it with trec_eval [13]. We rescored the predicted result sets of all eight published 2026 entries on CK26 and DB26 and compared every per-question score and every average with the official files: all 5,024 numbers agree exactly. The check is a script in the artifact.

Quirks of the official metric. (1) An ASK answer is scored as one document named true with relevance 1 or 0. A gold true therefore scores 1.0 whatever the system answers, and a gold false scores 0.0 for every system. (2) Every returned value has the same score, so trec_eval orders values by their text, and the official nDCG ignores the order the system returned. (3) The official F_{ndcg} average counts the mean nDCG of the scope once more as an extra item. Our official column reproduces all three; the strict column scores Booleans by exact match, uses the returned order for nDCG, and takes a plain mean.

Engine dependence of the CK gold answers. Of the seven CK26 gold queries that do not reproduce on Oxigraph, two break ties under LIMIT differently, two use xsd : int(), which is not a SPARQL 1.1 function, two format computed numbers differently (*e.g.*,3 against 3.0), and one computes a different value for an arithmetic expression. On rdflib, 42 of 50 reproduce.

D. Prompts

Figure 4 is the action catalogue and the rules shown to the planner at every turn, followed by the vocabulary, the linked entities, the history of actions and observations, the plan state, and the current query. Hermes$^-$ uses the same catalogue without the goal action, no plan state, and shorter rules: use only the terms shown or found, look entities up before using them, project only the answer variables, and finish when the result looks right.

```
State the goal first:
  {"op": "goal", "answer": ["?c"], "kind": "entity", "type": "ex:City", "many": true}
  kind is entity, literal, number or boolean; type is optional; many is false for one answer.
Actions. Reply with exactly one JSON object per turn.
Read the graph (the plan does not change):
  {"op": "lookup", "text": "Lake Placid"}             entities whose label contains the text
  {"op": "inspect", "iri": "ex:City"}                 predicates around an entity or a class
Change the plan (SPARQL algebra):
  {"op": "triple", "pattern": "?c ex:hosted ?g"}      add a triple pattern (join)
  {"op": "filter", "expr": "?year > 1900"}
  {"op": "optional", "patterns": ["?c ex:country ?k"]}
  {"op": "union", "branches": [["?x a ex:A"], ["?x a ex:B"]]}
  {"op": "minus", "patterns": ["?g ex:season \"Summer\""]}
  {"op": "bind", "expr": "YEAR(?date)", "var": "?y"}
  {"op": "group", "vars": ["?c"]}
  {"op": "aggregate", "expr": "COUNT(DISTINCT ?g)", "alias": "?n"}
  {"op": "having", "expr": "?n > 2"}
  {"op": "order", "expr": "DESC(?n)"}
  {"op": "limit", "n": 1}
  {"op": "offset", "n": 1}
  {"op": "project", "vars": ["?c"]}
  {"op": "distinct"}
  {"op": "ask"}                                       yes/no questions
  {"op": "remove", "text": "?c ex:country ?k"}        drop a triple pattern or filter
  {"op": "undo"}
  {"op": "finish"}                                    when the query answers the question

Rules: state the goal first. A verifier checks every step before it runs and
rejects ill-typed ones with a message; read it and fix the step. Use only the
prefixes and terms shown or found with lookup and inspect. Look entities up
before using them. Finish only when the plan state says goals accomplished.
```

Figure 4: Planner prompt: action catalogue and rules.